VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum – 590014

Report on

"Disaster Response System using Social Media Data"

Project Based Assignment

Submitted in partial fulfilment of internal assessment for the 6th Semester

For the subject 'Big Data Analytics (BAD601)'

Bachelor of Engineering
In
Computer Science Engineering (Data Science)
2025 - 2026

Submitted by

Pranav Kumar S A	**1AY23CD045**
Pratap H	**1AY23CD046**
Prateek Kumar Bal	**1AY23CD047**
Pratheek R S	**1AY23CD048**

Under the guidance of

Ms. Surbhi
Assistant Professor
Department of AIML and CSE (DS)

Department of
Computer Science Engineering
(Data Science)

Acharya Institute of Technology

(An Institute Affiliated to Visvesvaraya Technological University, Belgaum, Recognized by AICTE)
Acharya Dr. Sarvepalli Radhakrishnan Road, Soladevanahalli, Bangalore-560107

Index

Abstract

The rapid growth of social media platforms has enabled people to share information about disasters and emergencies in real time. During natural disasters such as floods, earthquakes, cyclones, and fires, social media data can provide valuable insights for emergency response and disaster management. However, handling and analyzing such large-scale streaming data using traditional systems is difficult due to high data volume, velocity, and variability.

This project presents a Big Data based Disaster Response System using social media data. The system is designed to detect disaster-related information from social media streams and generate alerts for affected regions. A disaster tweet dataset was used to simulate real-time social media streaming. Hadoop Distributed File System (HDFS) was used for distributed storage of tweet logs, while Apache Spark was utilized for real-time processing and filtering of disaster-related content. Hive was integrated for analytical querying within the Hadoop ecosystem, and MongoDB was used to store generated disaster alerts in a flexible NoSQL format. Link analytics and graph visualization techniques were also implemented to analyze relationships between disasters and affected locations.

Introduction

Natural disasters such as floods, earthquakes, cyclones, storms, and fires cause significant damage to human life, infrastructure, and the environment every year. Rapid detection and timely response during such events are essential to minimize losses and improve disaster management efficiency. In recent years, social media platforms have emerged as important sources of real-time information during emergencies, as users frequently share updates, images, and reports about ongoing disaster situations.

The large volume and continuous flow of social media data make traditional data processing systems inefficient for real-time analysis. Big Data technologies provide scalable and distributed solutions capable of handling massive datasets with high speed and reliability. Technologies such as Hadoop, Spark, Hive, and MongoDB enable efficient storage, processing, querying, and analysis of large-scale streaming data.

This project focuses on developing a Disaster Response System using social media data and Big Data technologies. Apache Spark is used for real-time stream processing, while MongoDB stores generated alerts for quick access and analysis.

System Architecture

- The system is designed using a distributed Big Data architecture for handling large-scale disaster-related social media data.
- Disaster tweet data is used to simulate real-time social media streaming. Apache Spark Streaming is used for continuous real-time processing of incoming tweet data.
- Disaster-related tweets are filtered using keyword-based distributed processing techniques. Hadoop Distributed File System (HDFS) is used for scalable and fault-tolerant storage of tweet logs.
- Distributed processing concepts based on the MapReduce paradigm are applied for filtering relevant disaster information.
- Hive is integrated to support SQL-like analytical querying on disaster data stored within the Hadoop ecosystem. MongoDB is used as a NoSQL database to store generated disaster alerts in JSON document format. Link analytics and graph visualization techniques are used to analyze relationships between disasters and affected locations.

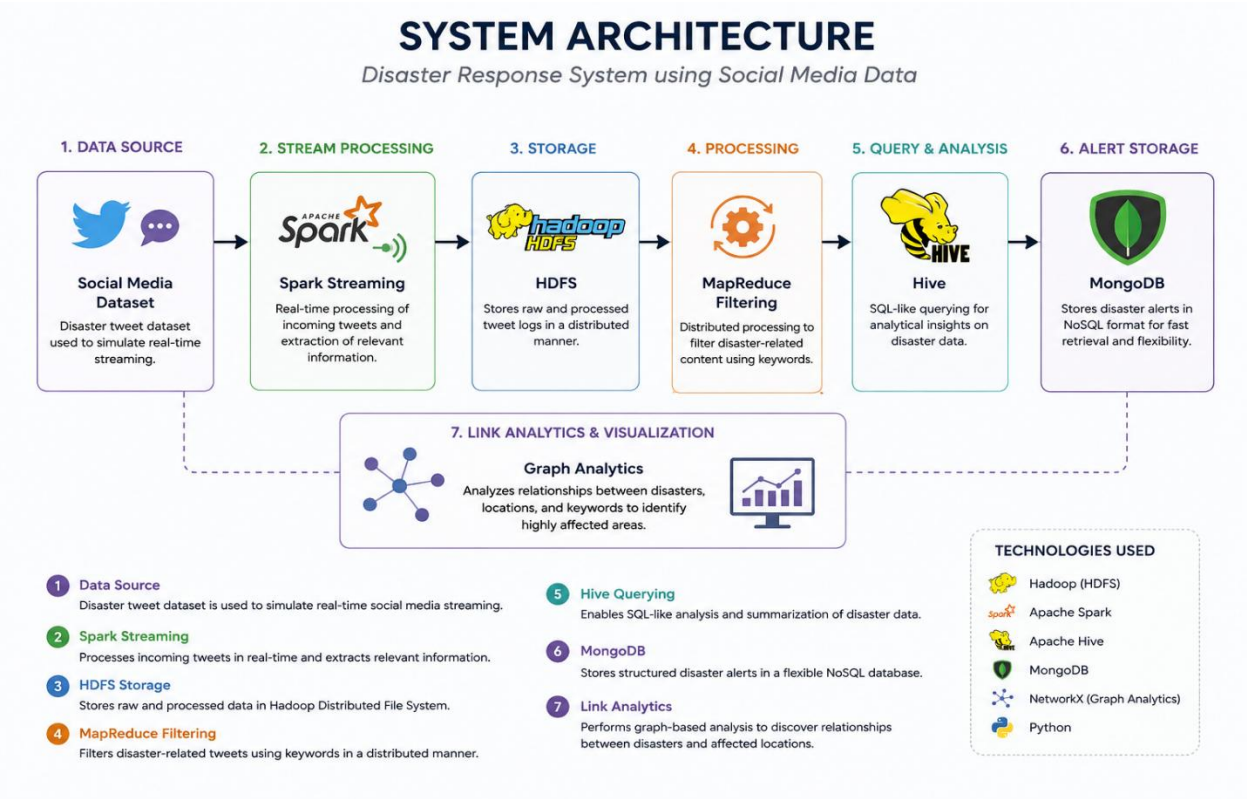

Fig 1. System Architecture

Technologies Used

Table 1. Software and tools Used

Technology / Tool	Purpose in the Project
Apache Hadoop	Provides the Big Data framework for distributed storage and processing.
HDFS (Hadoop Distributed File System)	Stores disaster tweet logs in a scalable and fault-tolerant distributed environment.
Apache Spark	Performs real-time stream processing and disaster tweet analysis.
Spark Streaming	Simulates continuous real-time tweet processing and alert generation.
Apache Hive	Supports SQL-like querying and analytical processing of disaster-related data.
MongoDB	Stores generated disaster alerts in flexible NoSQL document format.
Python	Used for scripting, data processing, Spark integration, and analytics implementation.
PySpark	Enables Python integration with Apache Spark for distributed processing.
NetworkX	Performs graph-based link analytics between disasters and affected locations.
Matplotlib	Generates graphical visualizations and analytics charts.
WSL (Windows Subsystem for Linux)	Provides Linux environment for running Hadoop ecosystem tools on Windows. Development and execution environment for scripts and commands.
Kaggle Disaster Tweets Dataset	Dataset used to simulate real-time social media disaster data.

Dataset Description

The dataset used in this project is the "Natural Language Processing with Disaster Tweets" dataset obtained from Kaggle. The dataset contains disaster-related and non-disaster-related tweets collected from social media platforms. It is commonly used for disaster detection and text classification tasks.

The dataset is used to simulate real-time disaster tweet streaming for processing and analysis using Big Data technologies.

Table 2. Dataset Features

Attribute	Description
id	Unique identifier for each tweet
keyword	Disaster-related keyword associated with the tweet
location	Location information mentioned in the tweet
text	Actual tweet content
target	Indicates whether the tweet is related to a real disaster (1) or not (0)

Dataset Characteristics

- The dataset contains thousands of disaster-related tweets.
- Tweets include information about floods, earthquakes, fires, cyclones, storms, and other emergency events.
- Both relevant and irrelevant tweets are included, making the dataset suitable for filtering and classification tasks.
- The dataset is semi-structured and text-intensive in nature.

Source

Kaggle – Natural Language Processing with Disaster Tweets Dataset.

Methodology (Implementation)

The proposed Disaster Response System follows a step-by-step Big Data processing workflow for handling disaster-related social media data. Multiple Big Data technologies are integrated to perform distributed storage, real-time processing, analytical querying, alert generation, and visualization.

1. Data Collection

A disaster tweet dataset obtained from Kaggle is used as the primary data source. The dataset contains tweets related to disasters such as floods, earthquakes, fires, cyclones, and storms. The dataset is used to simulate real-time social media streaming.

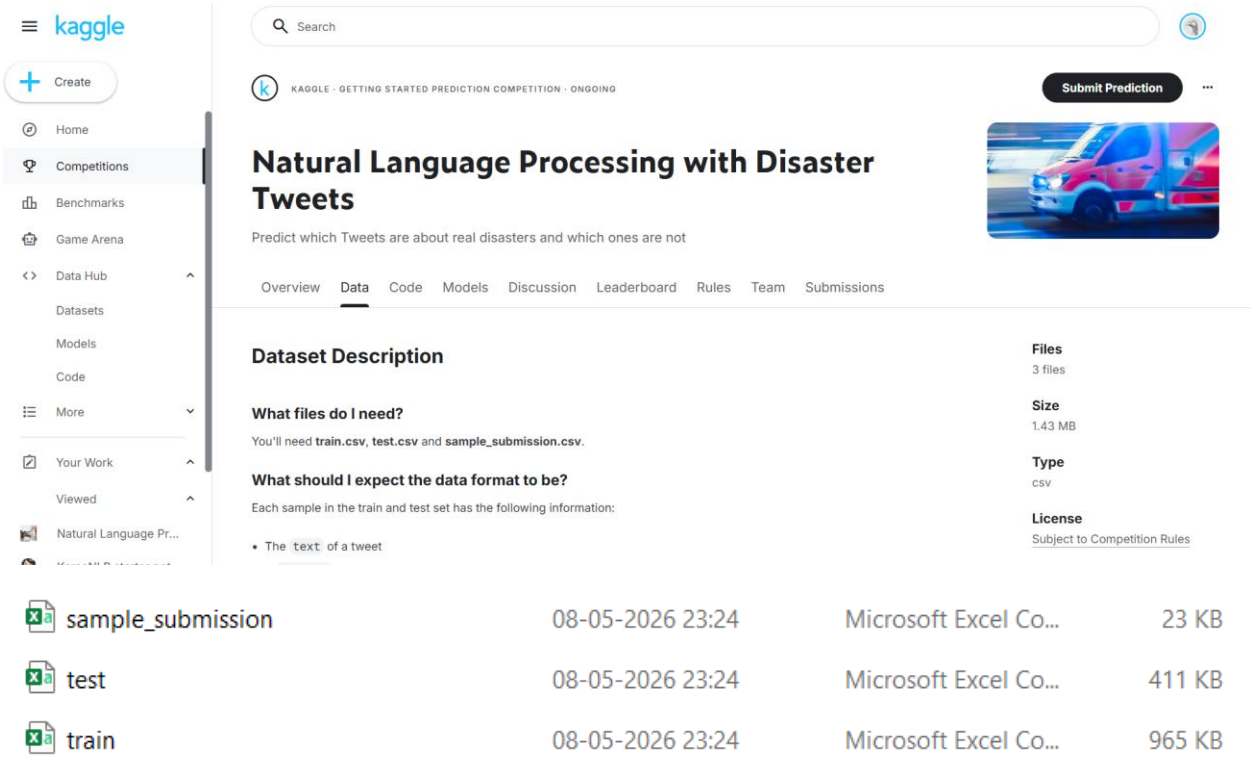

Fig 2. Dataset Description

2. Real-Time Stream Processing using Spark

Apache Spark Streaming is used to process incoming tweet data continuously. Tweets are read one-by-one to simulate live social media feeds. Disaster-related keywords are identified using keyword filtering techniques.

Keywords used for filtering include:

- Flood
- Earthquake
- Fire
- Cyclone
- Storm

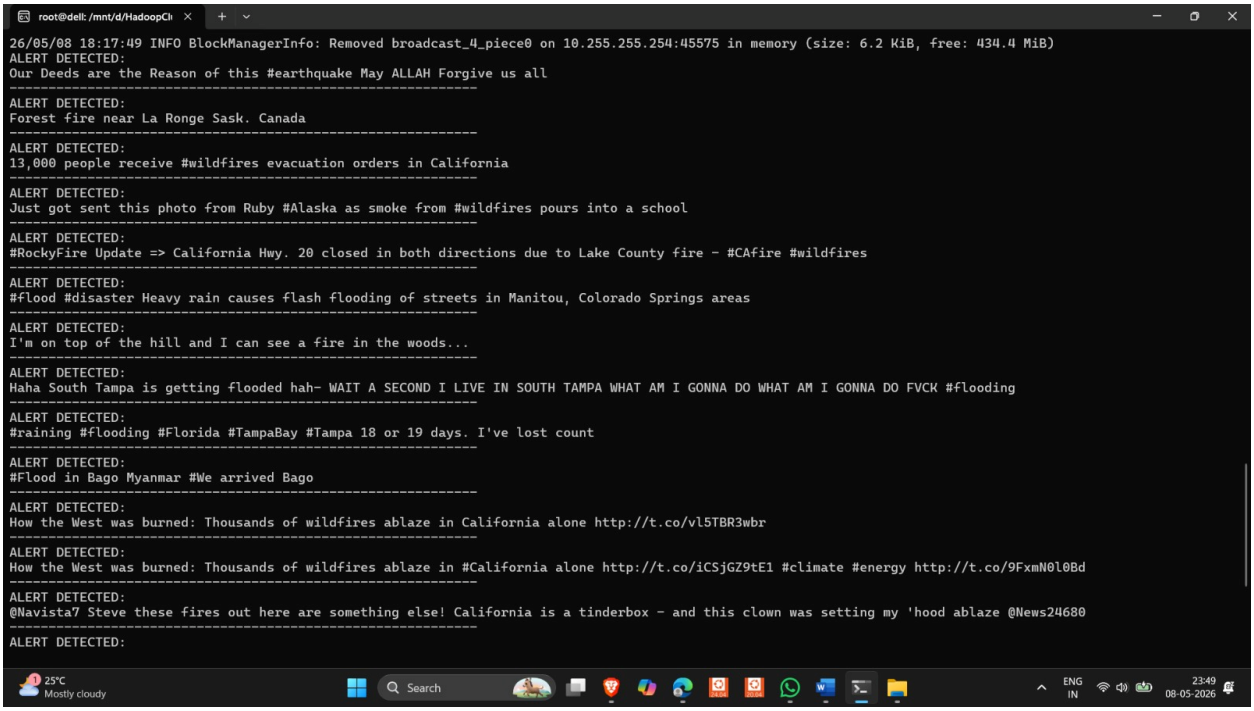

Fig 3. Stream Processing using PySpark

3. HDFS Storage

The disaster tweet dataset and processed logs are stored in Hadoop Distributed File System (HDFS). HDFS provides distributed and fault-tolerant storage for large-scale data processing.

The dataset is uploaded into HDFS directories for distributed access and management.

4. Distributed Filtering using MapReduce Concepts

Relevant disaster tweets are filtered using distributed processing logic following the MapReduce paradigm. Tweets containing disaster-related keywords are identified and processed for alert generation and analysis.

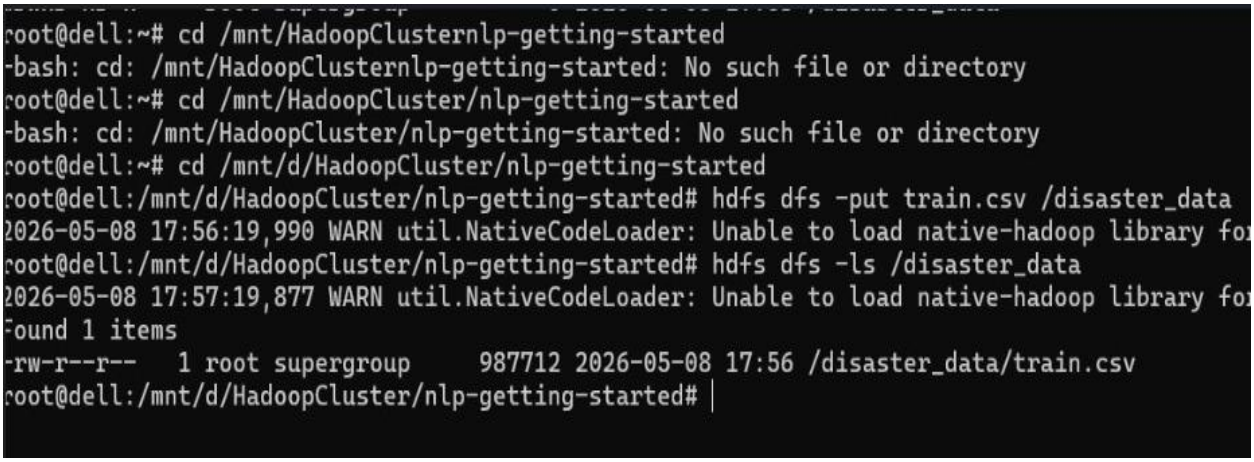

Fig 4. HDFS directory disaster_data containing train.csv

5. Hive Integration and Querying

Apache Hive is integrated into the Hadoop ecosystem to support SQL-like querying and analytical operations on disaster-related data.

Hive enables structured analysis and summarization of tweet information stored in the distributed environment.

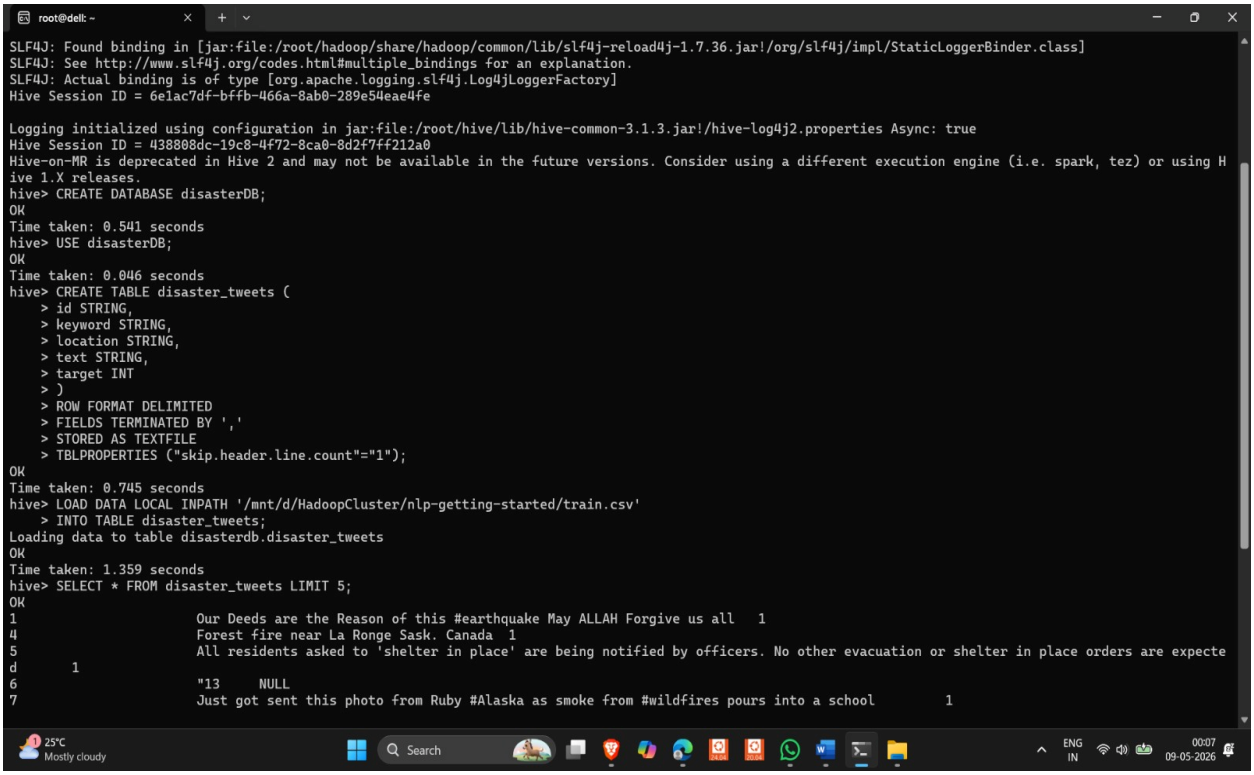

Fig 5. Hive Queries for tweet analysis and summarization

6. MongoDB Alert Storage

Filtered disaster alerts are stored in MongoDB using JSON document format. Each alert contains:

- Disaster type
- Location
- Severity level
- Tweet content

MongoDB provides flexible NoSQL storage for semi-structured real-time alert data.

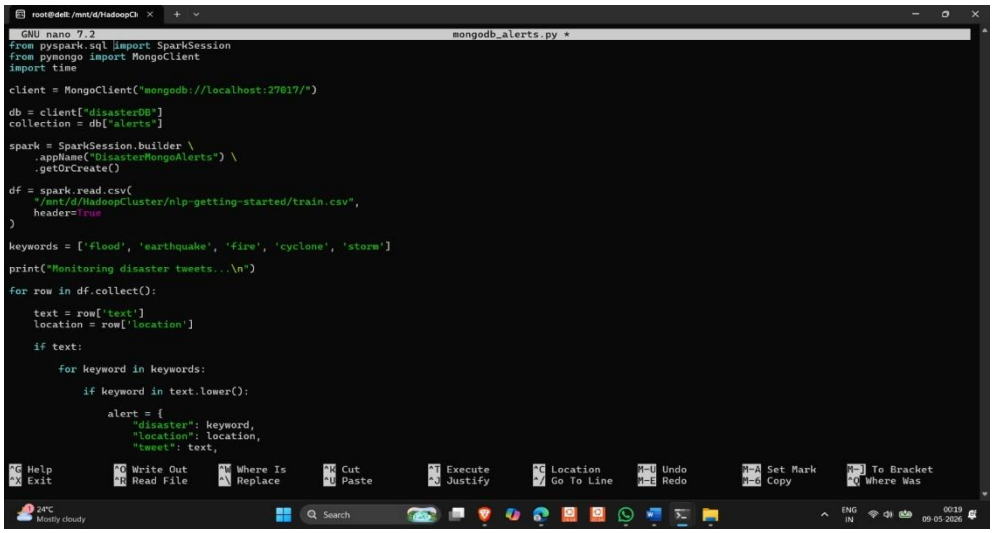

Fig 6. Mongo_alerts.py for save alerts in MongoDB Collections

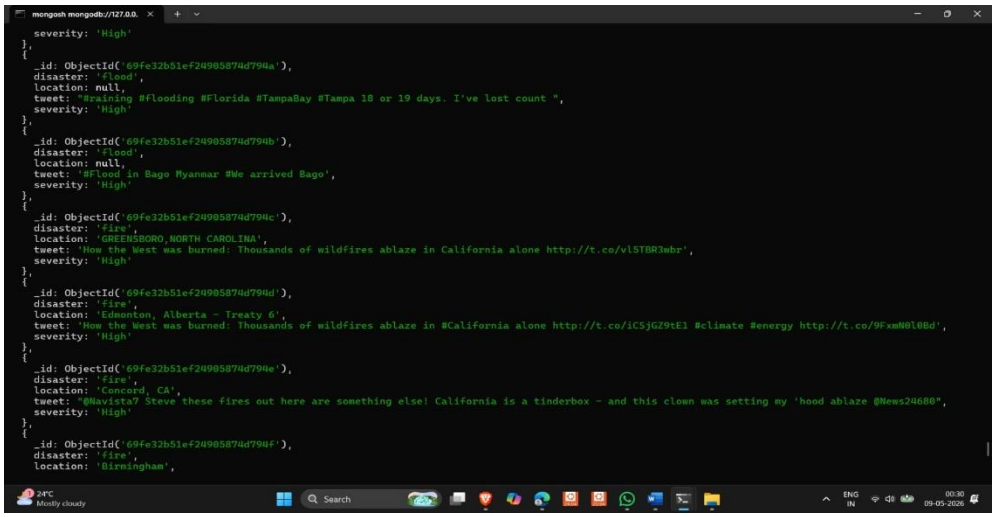

Fig 7. Alerts saved in disasterDB Collection

7. Link Analytics and Visualization

Graph-based link analytics is performed using NetworkX and Matplotlib libraries. Relationships between disasters and affected locations are visualized using graph structures to identify highly impacted regions and disaster patterns.

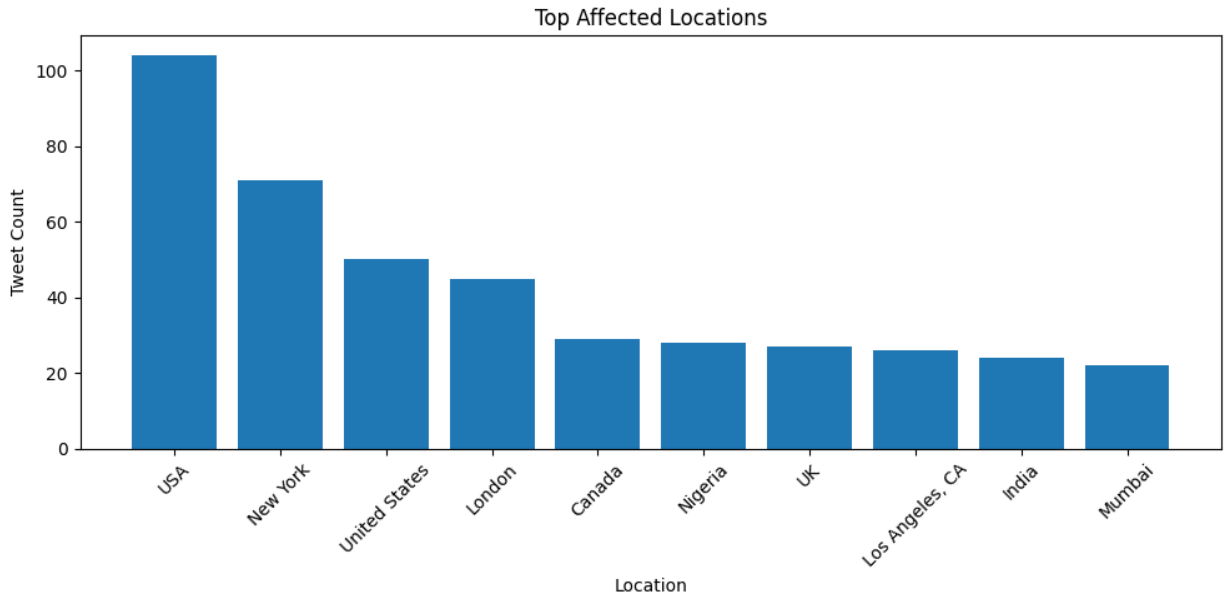

Fig 8. Tweets from various countries

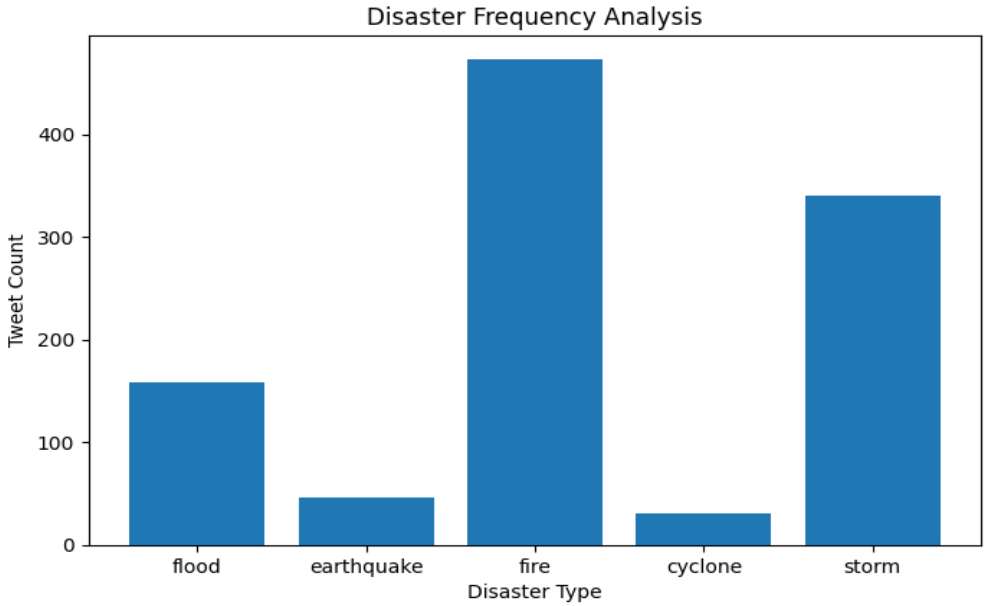

Fig 9. Frequency (occurrences) of different disaster types

Results

The Disaster Response System using Social Media Data was successfully implemented using multiple Big Data technologies including Hadoop, Spark, Hive, MongoDB, and graph analytics tools. The system was capable of processing disaster-related tweet data, generating alerts, and visualizing affected locations.

- Disaster tweets were successfully processed using Apache Spark Streaming.
- Disaster-related tweets were filtered using keyword-based analysis.
- Tweet logs were stored in Hadoop Distributed File System (HDFS).
- Disaster alerts were stored in MongoDB using JSON format.
- Link analytics and multiple visualizations were generated successfully.

Table 3. Advantages, Limitations and Future Enhancements

Advantages	Limitations	Future Enhancements
Supports scalable distributed storage and processing of large-scale social media data.	Uses a pre-collected dataset instead of live social media APIs.	Integrate live Twitter/X API for real-time disaster data collection.
Enables near real-time disaster monitoring and alert generation.	Real-time streaming is simulated rather than fully live.	Implement advanced NLP and machine learning models for intelligent disaster detection.
Provides efficient stream processing using Apache Spark Streaming.	Implemented on a single-node Hadoop environment.	Add sentiment analysis and severity prediction mechanisms.
Supports flexible NoSQL storage using MongoDB.	Disaster detection is primarily keyword-based.	Develop interactive dashboards and GIS-based heatmap visualizations.
Integrates multiple Big Data technologies into a unified analytics pipeline.	Hive metastore instability may occur in WSL environments.	Deploy the system on cloud platforms for scalable real-world applications.

Conclusion

The “Disaster Response System using Social Media Data” successfully demonstrated the practical implementation of a Big Data based disaster monitoring and analytics pipeline using modern distributed technologies. The project focused on processing disaster-related social media information efficiently through scalable storage, real-time stream processing, analytical querying, NoSQL alert management, and graph-based visualization techniques.

With the rapid growth of social media platforms, disaster-related information is generated continuously in massive volumes. Traditional systems often face challenges in processing such large-scale streaming data efficiently and in real time. The proposed system addressed these challenges by integrating multiple Big Data technologies including Hadoop, HDFS, Spark Streaming, Hive, MongoDB, and graph analytics tools into a unified architecture capable of handling disaster-related tweet data effectively.

Apache Hadoop and HDFS were used to provide scalable and fault-tolerant distributed storage for disaster tweet logs. Apache Spark Streaming enabled near real-time tweet processing and disaster keyword filtering, allowing the system to simulate continuous disaster monitoring. Hive and Spark SQL were integrated to support analytical querying and structured data analysis within the Hadoop ecosystem. MongoDB provided flexible NoSQL storage for disaster alerts generated during processing, while graph analytics and visualization techniques were used to identify relationships between disasters and affected locations.

The project also demonstrated the importance of data analytics and visualization in disaster response systems. Multiple analytical outputs including bar charts, pie charts, trend graphs, word clouds, and link analytics graphs were generated to provide meaningful insights into disaster patterns, frequently affected regions, and disaster-related social media activity. These visualizations improved the interpretability of the processed data and enhanced overall analytical capabilities.

Overall, the proposed system successfully achieved its objectives and demonstrated the effectiveness of integrating distributed computing, stream processing, NoSQL databases, and analytical visualization techniques for real-time disaster monitoring and response analysis using social media data. The project further emphasizes the growing role of Big Data technologies in solving real-world emergency management and disaster response challenges.

Team Member Contributions

Table 4. Individual Contributions

Team Member	Contribution
Pranav	Configured Hadoop ecosystem, managed HDFS setup, uploaded dataset to distributed storage, and handled Hadoop cluster operations.
Pratap	Implemented Apache Spark Streaming for real-time disaster tweet processing and keyword-based filtering mechanisms.
Prateek	Integrated MongoDB for disaster alert storage and managed JSON-based NoSQL database operations.
Pratheek	Developed link analytics visualizations, generated analytical graphs and charts, and prepared project documentation and presentation materials.

Collaborative Contributions

- All team members participated in system design, testing, debugging, and integration of Big Data technologies.
- Team members collectively contributed to report preparation, result analysis, screenshot collection, and final project presentation.
- The project was developed through coordinated teamwork involving distributed storage, real-time processing, database management, and analytical visualization modules.

Team Coordination and Workflow

- Work was divided based on individual technical modules to ensure efficient project development.
- Regular discussions and integration sessions were conducted to combine all components into a unified disaster response pipeline.
- All modules were tested collectively to ensure smooth communication between Hadoop, Spark, MongoDB, and analytics components.

References

1. Apache Hadoop Documentation. Apache Software Foundation. Available at: https://hadoop.apache.org/docs/
   Accessed for understanding Hadoop ecosystem configuration, HDFS architecture, distributed storage mechanisms, and cluster management concepts used in the project.
2. Apache Spark Documentation. Apache Software Foundation. Available at: https://spark.apache.org/documentation.html
   Referred for implementing Spark Streaming, real-time data processing workflows, and distributed analytical operations using PySpark.
3. Apache Hive Documentation. Apache Software Foundation. Available at: https://hive.apache.org/
   Used for studying Hive architecture, SQL-like querying mechanisms, schema management, and analytical querying integration within the Hadoop ecosystem.
4. MongoDB Official Documentation. MongoDB Inc. Available at: https://www.mongodb.com/docs/
   Referred for implementing NoSQL-based disaster alert storage using JSON document structures and database query operations.
5. PySpark API Documentation. Apache Spark. Available at: https://spark.apache.org/docs/latest/api/python/
   Used for understanding Python integration with Apache Spark and implementing distributed stream processing operations.
6. Kaggle – Natural Language Processing with Disaster Tweets Dataset. Available at: https://www.kaggle.com/competitions/nlp-getting-started
   The primary dataset source used for simulating real-time disaster-related social media tweet streams and performing analytical processing.
7. NetworkX Documentation. Available at: https://networkx.org/documentation/stable/
   Used for implementing graph-based link analytics and visualization of relationships between disaster events and affected regions.